Reinforcement Learning for Congestion Control: A Review of the 50 Most-Cited Studies

Gaurab Baral Nityashree Vangala

CS 6043: Computer Networking, Fall 2026

1 Introduction

Congestion control (CC) decides how fast a sender may push data into a shared network. After the Internet's congestion collapses of 1986, Jacobson's slow start and additive-increase/multiplicative-decrease rules stabilized the network by backing off when packets were lost [1]. Four decades later the problem takes many forms: gigabit fiber, Wi-Fi, cellular links whose capacity changes every few milliseconds, satellite paths, and data centers where a round trip takes tens of microseconds. Each setting has produced its own hand-designed algorithms, such as CUBIC [2] and BBR [3] for the Internet and DCTCP [4] for data centers.

Reinforcement learning (RL) offers a different route: the controller learns its control law from experience instead of having it written by hand. Remy [5] and PCC [6] showed that machine-generated or online-learned controllers can beat hand-tuned ones, and Aurora [7] cast Internet CC explicitly as a deep RL problem in 2019. Since then the field has grown quickly. Yet learned controllers that look excellent in a simulator sometimes fail on links they never saw, few run inside an operating-system kernel, and fewer still have been deployed.

This report asks three research questions:

RQ1 **Design.** What do RL congestion controllers observe, what do they change, and what reward do they learn from?

RQ2 **Testing.** How are they trained and tested, and do they work on networks they did not see in training?

RQ3 **Deployment.** What stops operators from using them, and what research could fix that?

We followed the PRISMA 2020 reporting guideline [8]. A scripted search of OpenAlex, arXiv and USENIX NSDI/ATC for 2016–2026, with forward and backward citation searching and a broadened update search, identified 2,459 records. After removing duplicates and screening titles, abstracts and 427 full texts, 362 primary studies (in 396 papers) were eligible. Because of time constraints we review the 50 of them with the most citations in a single Semantic Scholar snapshot (9 November 2026); the cutoff is 27 citations. For each study we coded the setting, control point, RL method, state, action, reward, the highest evaluation level reached (simulation, emulation or real network), and whether it tested generalization, evaluated fairness and released code. We call a design *hybrid* when a classic algorithm or safeguard remains in the deployed control loop.

Our main observations are that the most-cited designs increasingly wrap the learned part around a classic algorithm, that fewer than half reach a real network, that few test outside their training distribution or release code, and that every deployed system is run by an operator that owns the endpoints and optimizes one narrow service metric.

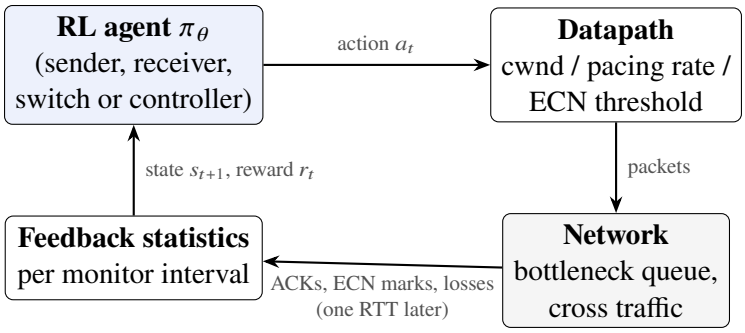


Figure 1: The usual RL congestion-control loop. The agent acts once per monitor interval and sees the consequences one RTT later.

2 Detailed Description

2.1 Background: congestion control and reinforcement learning

A congestion controller should fill the bottleneck, avoid standing queues and loss, share capacity fairly, converge quickly, and coexist with the deployed base; Ware et al. argue that the last property is best judged by the *harm* a new algorithm does to existing traffic [9]. End-to-end controllers infer congestion from packet loss, round-trip time (RTT), delivery rate and Explicit Congestion Notification (ECN) marks, and they act through the *congestion window* (cwnd), which caps the data in flight, or the *pacing rate*. CUBIC grows cwnd as a cubic function of the time since the last loss [2]; BBR estimates the bottleneck bandwidth and minimum RTT and paces at that bandwidth [3]; DCTCP reacts to the fraction of ECN-marked packets [4].

RL models sequential decisions as a Markov decision process: at each step the agent observes a state s_t, takes an action a_t from its policy π, receives a reward r_t, and seeks to maximize the expected discounted return $\mathbb{E}[\sum_t \gamma^t r_t]$ [10]. The 50 studies use value-based methods (Q-learning and deep Q-networks), policy-gradient methods such as PPO [11], off-policy actor-critic methods such as TD3 [12], offline RL on logged data, imitation of an expert, contextual bandits, and multi-agent RL.

2.2 Congestion control as an RL problem

Figure 1 shows the formulation that Aurora made popular. Time is split into *monitor intervals*; the agent acts at the start of each, and when the ACKs return the sender computes statistics and a reward. Designs differ in where the agent runs (sender, receiver, switch or central controller), what it observes (usually a short history of throughput, RTT, RTT gradient and loss), what it sets (cwnd, rate, a parameter of a classic algorithm, or an ECN threshold), and how the reward weighs throughput against delay and loss. Aurora, for example, uses $10 \cdot \text{throughput} - 1000 \cdot \text{latency} - 2000 \cdot \text{loss}$. The reward fixes the operating point, so it is in effect the controller's specification. Because the result of an action arrives one RTT later, mixed with the effects of other flows, credit assignment is noisy, and because the controller runs on networks unlike any training environment, performance under distribution shift is a central measure of quality.

Table 1: Landmark designs among the 50. Level: highest evaluation environment (Emu = emulation).

Study	Action	Learning signal	Algorithm	Level
Indigo [13]	cwnd step every 10 ms	imitate an oracle that knows the emulated link's ideal cwnd	imitation, LSTM	Real
Aurora [7]	rate multiplier per interval	weighted throughput, latency, loss	PPO	Emu
Orca [14]	scale CUBIC's cwnd by 2^{2a}	throughput minus loss, scaled by delay	TD3	Real
MOCC [15]	rate	preference-weighted throughput, delay, loss	PPO	Real
Genet [16]	(training method)	Aurora's reward with a curriculum	PPO	Real
Sage [17]	cwnd	logged behavior of existing heuristics	offline RL	Real
Astraea [18]	rate	throughput, delay, loss, fairness	multi-agent	Real
Mutant [19]	select a kernel CC scheme	power (throughput / delay)	LinUCB bandit	Real

2.3 Design families among the 50

Of the 50 studies, 36 target the wide-area Internet or a generic wired bottleneck, 6 wireless or cellular networks, 4 IoT or sensor networks, 3 data centers and 1 satellite links. Of the 43 design studies, 33 run the agent at the sender. Table 1 summarizes the designs that later work builds on most.

Early Q-learning controllers. The earliest designs among the 50 use Q-learning over small, hand-quantized state spaces. QTCP approximates the Q-function with Kanerva coding and reports 59.5% higher throughput than rule-based TCP [20].

Clean-slate deep RL. Aurora [7] set the formulation most later work adopts. Trained with PPO on simulated links of 1.2–6 Mbit/s, it kept near-full utilization in emulation from 1 to 128 Mbit/s, but performed poorly at 1 ms latency, where packet-processing noise absent from training dominated the latency signal. Indigo, released with the Pantheon testbed [13], imitated an oracle that knew each emulated link's ideal window. MVFST-RL [21] runs the policy asynchronously inside a QUIC stack, because in a real stack the network does not pause while the agent computes.

Hybrids on top of classic control. Hybrids make up 15 of the 43 design studies (35%). They let a classic algorithm react per ACK and let the agent adjust it more slowly. In Orca [14], CUBIC runs in the kernel and, once per monitoring period, a TD3 agent multiplies its window by 2^{2a}, $a \in [-1, 1]$; it performed more consistently than clean-slate controllers at lower overhead. Spine [22] has a coarse agent emit sub-policies for a light executor, cutting control overhead by at least 80%. Two designs instead learn *which* algorithm to use: TCP-RL [23] sets the initial window and selects among 14 algorithms on a large search engine's web servers, and Mutant [19] switches among kernel protocols with a contextual bandit.

Learning from logs, multiple objectives and fairness. Sage [17] trains offline on trajectories from existing heuristics and outperforms the heuristics it observed. Concerto [24] imitated more than one million logged video sessions. A single reward fixes one operating point, so MOCC [15] conditions a PPO policy on a preference vector and adapts to new objectives 14.2 times faster than

Table 2: The 50 most-cited studies against the other 312 eligible studies. Generalization, fairness, code and hybrid use design studies as the denominator; evaluation shares use studies whose level is known.

	Top 50	Other 312
Design studies	43	282
Reach a real network	18/42 (43%)	39/204 (19%)
Simulation only	13/42 (31%)	131/204 (64%)
Test outside training distribution	7 (16%)	28 (10%)
Evaluate fairness	12 (28%)	51 (18%)
Release code	7 (16%)	18 (6%)
Hybrid (classic part in the loop)	15 (35%)	78 (28%)
First reported 2023 or later	4 (8%)	218 (70%)
Median practice score (0–5)	2	1

prior work. Copies of a policy trained alone tend to be greedy at a shared bottleneck; Astraea [18] trains flows jointly with a fairness term and converges up to 8.4 times faster to near-optimal fairness.

Other settings. In real-time video the controller is a bandwidth estimator: Loki [25] fuses an RL model with a rule-based one, and OnRL [26] learns online from mobile users, with a rule-based safeguard that takes over when the policy misbehaves. In data centers, ACC [27] runs an agent on every switch to tune ECN thresholds, and RL-CC [28] treats RDMA congestion control as a multi-agent problem. All IoT studies with a known evaluation level stop at simulation.

2.4 How the studies are trained and tested

We distinguish evaluation levels by how much of the real system they include. *Simulation* (ns-3, OMNeT++ or custom models) leaves out the operating system's stack, including timers, pacing and ACK aggregation. *Emulation* runs real stacks over software-imposed links, for example with Mahimahi or Mininet. *Testbeds and real networks*, such as Pantheon's globally distributed nodes [13], cloud paths and production traffic, expose the controller to real links and traffic. Four of the 50 are frameworks for this purpose: Pantheon, MVFST-RL, OpenNetLab and Iroko, whose authors report instability and overfitting to the training topology. Two others verify trained policies formally, including Aurora; whiRL proves some requirements and finds counterexamples for others [29].

3 Results

3.1 Evaluation evidence

Table 2 compares the 50 with the other 312 eligible studies. Of the 48 studies with an empirical evaluation, 18 reach a real network, 11 stop at emulation, 13 stop at simulation and 6 do not report a level.

Observation 1: the most-cited studies test on real networks more often, but still rarely. They reach a real network more than twice as often as the rest and rely on simulation alone about half as

Table 3: Studies among the 50 reporting production use or tests with real users.

System	Setting	Learned component	Classic in loop	Reported scale
TCP-RL [23]	Web search	initial window; algorithm selection	yes	search engine, over a year; 23% shorter short flows
Concerto [24]	Video calls	bitrate policy imitated from logs	no	production video-telephony app
OnRL [26]	Mobile video	per-user online RL, aggregated model	yes	543 h, 151 users; 14.2% fewer stalls
Loki [25]	Real-time video	RL fused with a rule-based model	yes	101 million sessions; 26–44% fewer stalls
ACC [27]	Data center	per-switch ECN threshold	yes	300 machines; 30% lower flow completion time

often. Part of this gap reflects how much we could read: the top 50 were coded from full text about twice as often, and abstracts omit evaluation details.

Observation 2: citations do not guarantee strong evidence. Each design study gets one point for each of five practices: reporting the evaluation environment, using emulation or a real network, testing generalization, evaluating fairness, and releasing code. The median top design scores 2 of 5. Only 6 score four or more: Orca scores 5, and Aurora, Genet, MOCC, Astraea and Mutant score 4.

Observation 3: generalization is tested rarely, and broad training is what helps. Only 7 of 43 designs test outside their training range, and only 7 release code, which limits what later work can compare against. Where generalization succeeds, it comes from deliberately broad training or data from the deployment itself. Aurora handled bandwidths more than 20 times above its training range but not unseen latency noise. Genet [16] trains preferentially on environments where the RL policy lags rule-based baselines, and its policies beat regularly trained RL and rule-based baselines in real environments. OnRL adapts online from user sessions.

Observation 4: independent evaluations are less favorable. Recent independent tests fall outside the 50, probably because they have had little time to gather citations. Giacomoni and Parisis find that public RL controllers grab bandwidth quickly and tolerate random loss, but cause excess loss, are unfair in almost all setups, and lag when conditions change [30]. Ghermezian et al. ran an Aurora-style controller on a 10 Gbit/s wide-area path, where it reached 0.5% mean utilization while CUBIC and BBR reached up to 9.9 and 9.7 Gbit/s [31]. A reader who sees only the most-cited work would miss these results.

3.2 Deployment

Table 3 lists the 5 studies among the 50 that report production use or tests with real users.

Observation 5: deployed systems share a pattern. Each is run by an organization that controls the sender or the switches and can therefore monitor, test and roll back. Each optimizes one narrow service metric: transfer time, stalls or storage throughput. Four of the five keep a classic algorithm or safeguard in the loop, and the fifth, Concerto, learned from logged sessions with no online exploration. None is a general-purpose controller for Internet traffic.

Barriers. Deployments sidestep *generalization* by training on their own traffic or by limiting what a wrong decision can do. *Safety* relies on runtime fallbacks (OnRL) rather than on verification, which so far checks trained policies after the fact [29]. *Fairness and coexistence* with CUBIC and BBR are evaluated by only 12 of 43 designs. *Overhead* pushes learning off the per-packet path: Orca and Spine separate timescales, and NVIDIA's follow-up to RL-CC distilled its policy into decision trees, cutting inference time 500-fold to fit on a network card [32]. *Lifecycle* questions, such as when to retrain, how to detect drift and how to roll back a model, are rarely discussed.

LLM-designed controllers. None of the 50 uses a large language model, since this line of work began around 2024. From our wider search, PolicySmith uses an LLM to generate heuristics that run in the Linux kernel [33], and Trailblazer invokes compact LLMs only in complex conditions and beat an optimized industrial policy in a large A/B test [34]. Both stay off the per-packet path, the same division of labor as Orca. Generated code can be reviewed like any heuristic, but a generator can overfit to the emulator that scores it.

Limitations. Citation counts favor older papers and prominent venues: only 4 of the 50 first appeared in 2023 or later, against 70% of the other studies. Only 19 of the 50 were coded from full text, so practices that abstracts rarely mention are likely undercounted. Screening and coding were done in a single pass with tool support and have not yet been duplicated independently.

4 Summary

We reviewed the 50 most-cited of 362 eligible studies on RL for congestion control.

- **Design (RQ1).** The studies observe short histories of throughput, delay and loss, act mainly on the window or the rate, and optimize a weighted reward. 15 of 43 designs keep a classic algorithm or safeguard in the deployed loop, and later designs increasingly learn from logs or bound the learned part.
- **Testing (RQ2).** 43% of studies with a known evaluation level reach a real network, more than the 19% of other studies. Only 16% of designs test outside their training range and 16% release code. Independent evaluations find public RL controllers unfair and brittle.
- **Deployment (RQ3).** All 5 deployed systems are run by operators who own the endpoints or switches and optimize a narrow service metric; four keep a classic component in the loop. Generalization, coexistence with the deployed base and model lifecycle remain open.

Promising directions are benchmarks with explicit training and test distributions, coexistence tests that measure harm to CUBIC and BBR [9], runtime assurance as a designed component, and learned

components built for kernel, QUIC or NIC datapaths from the start. Extending this review beyond the most-cited studies would also capture the post-2023 literature that citation ranking leaves out.

References

[1] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols*. ACM, Aug. 1988, pp. 314–329. [Online]. Available: http://dx.doi.org/10.1145/52324.52356

[2] S. Ha, I. Rhee, and L. Xu, "CUBIC: a new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008. [Online]. Available: http://dx.doi.org/10.1145/1400097.1400105

[3] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, no. 5, pp. 20–53, Oct. 2016. [Online]. Available: http://dx.doi.org/10.1145/3012426.3022184

[4] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 conference*. ACM, Aug. 2010, pp. 63–74. [Online]. Available: http://dx.doi.org/10.1145/1851182.1851192

[5] K. Winstein and H. Balakrishnan, "TCP ex machina: computer-generated congestion control," in *Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM*. ACM, Aug. 2013, pp. 123–134. [Online]. Available: http://dx.doi.org/10.1145/2486001.2486020

[6] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 395–408.

[7] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "A Deep Reinforcement Learning Perspective on Internet Congestion Control," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 3050–3059.

[8] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, p. n71, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1136/bmj.n71

[9] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Beyond Jain's Fairness Index: Setting the Bar For The Deployment of Congestion Control Algorithms," in *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2019, pp. 17–24. [Online]. Available: http://dx.doi.org/10.1145/3365609.3365855

[10] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[11] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[12] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1587–1596.

[13] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: The Training Ground for Internet Congestion-Control Research," in *2018 USENIX Annual Technical Conference (USENIX ATC)*, 2018.

[14] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic Meets Modern: a Pragmatic Learning-Based Congestion

Control for the Internet," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 632–647.

[15] Y. Ma, H. Tian, X. Liao, J. Zhang, W. Wang, K. Chen, and X. Jin, "Multi-objective congestion control," in *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, Mar. 2022, pp. 218–235.

[16] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang, "Genet: automatic curriculum generation for learning adaptation in networking," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 397–413.

[17] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control," in *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 2023, pp. 255–274.

[18] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards Fair and Efficient Learning-based Congestion Control," in *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, Apr. 2024, pp. 99–114.

[19] L. Pappone, A. Sacco, and F. Esposito, "Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.

[20] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive Congestion Control with Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2019.

[21] V. Sivakumar, O. Delalleau, T. Rocktäschel, A. H. Miller, H. Küttler, N. Nardelli, M. Rabbat, J. Pineau, and S. Riedel, "MVFST-RL: An Asynchronous RL Framework for Congestion Control with Delayed Actions," arXiv:1910.04054, 2019.

[22] H. Tian, X. Liao, C. Zeng, D. Sun, J. Zhang, and K. Chen, "Efficient DRL-Based Congestion Control With Ultra-Low Overhead," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 1888–1903, 2024.

[23] X. Nie, Y. Zhao, Z. Li, G. Chen, K. Sui, J. Zhang, Z. Ye, and D. Pei, "Dynamic TCP Initial Windows and Congestion Control Schemes Through Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1231–1247, 2019.

[24] A. Zhou, H. Zhang, G. Su, L. Wu, R. Ma, Z. Meng, X. Zhang, X. Xie, H. Ma, and X. Chen, "Learning to Coordinate Video Codec with Transport Protocol for Mobile Video Telephony," in *The 25th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2019, pp. 1–16.

[25] H. Zhang, A. Zhou, Y. Hu, C. Li, G. Wang, X. Zhang, H. Ma, L. Wu, A. Chen, and C. Wu, "Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based models," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2021, pp. 775–788.

[26] H. Zhang, A. Zhou, J. Lu, R. Ma, Y. Hu, C. Li, X. Zhang, H. Ma, and X. Chen, "OnRL: improving mobile video telephony via online reinforcement learning," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. ACM, 2020, pp. 1–14.

[27] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, and W. Deng, "ACC: automatic ECN tuning for high-speed datacenter networks," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 384–397.

[28] C. Tessler, Y. Shpigelman, G. Dalal, A. Mandelbaum, D. Haritan Kazakov, B. Fuhrer, G. Chechik, and S. Mannor, "Reinforcement Learning for Datacenter Congestion Control," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 2, pp. 43–46, Jan. 2022.

[29] T. Eliyahu, Y. Kazak, G. Katz, and M. Schapira, "Verifying learning-augmented systems," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 305–318.

[30] L. Giacomoni and G. Parisis, "Reinforcement Learning-based Congestion Control: A Systematic Evaluation of Fairness, Efficiency and Responsiveness," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. IEEE, May 2024, pp. 1451–1460.

[31] A. Ghermezian, K. Karpov, D. Kachan, V. Kirova, and E. Siemens, "PPO-Driven Reinforcement Learning Congestion Control Under High- BDP Wide-Area Deployment: A Scaling Analysis," 2026. [Online]. Available: https://opendata.uni-halle.de//handle/1981185920/125740

[32] B. Fuhrer, Y. Shpigelman, C. Tessler, S. Mannor, G. Chechik, E. Zahavi, and G. Dalal, "Implementing Reinforcement Learning Datacenter Congestion Control in NVIDIA NICs," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, May 2023, pp. 331–343.

[33] R. Dwivedula, D. Saxena, A. Akella, S. Chaudhuri, and D. Kim, "Man-Made Heuristics Are Dead. Long Live Code Generators!" in *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2025, pp. 51–60.

[34] D. Wu, L. Kang, Z. Wang, F. Wang, W. Zhang, C. Sun, X. Tao, W. Yang, L. Zhang, W. Zhu, P. Cui, and Z. Wang, "Grounding Large Language Models as Generalizable Policies in Network Control," arXiv:2512.11839, 2025.

Contributions of Group Members

Gaurab Baral: [to be filled in by the authors]
Nityashree Vangala: [to be filled in by the authors]